

Covariance Leakage in the NGCC Shuttle Submission

Equivalent-Key Recovery and Fresh-Message Forgery

Against All Three Submitted Parameter Sets

Martin Feussner
Selmer Center, University of Bergen
`martin.feussner@uib.no`

25 September 2026

Provenance and review status. The attack, derivation, implementation, experiments, and initial manuscript were produced independently by OpenAI Codex using Daybreak Blue at max reasoning effort during an AI-run audit of NGCC signature candidates. At the date above, no human cryptanalyst has independently verified the argument, implementation, measurements, or novelty assessment. This report is a preliminary disclosure for human review and reproduction.

Abstract

Shuttle is a lattice signature submitted to the Next-generation Commercial Cryptographic Algorithms Program. Its rejection-free iterative sampler is intended to select between $y - v$ and $y + v$ so that the response remains close to a secret-independent Gaussian. The submitted specification defines $p_v(y)$ as the probability of $y - v$, but its transition pseudocode applies the same event to $y + v$. The submitted reference implementation follows this reversed update. The authors’ independently published ePrint version uses the opposite, intended branch direction. This establishes a concrete mismatch between the NGCC submission and the separately published construction; the attack described here targets that NGCC-specific mismatch.

The reversal changes a hiding transition into an amplifying one. In the idealized exact-transition analysis, reversing the branch changes covariance cancellation into an additional rank-one term $2vv^T$. Shuttle’s first secret component is the public constant one, so the transmitted first response polynomial supplies an anchor for every nonzero challenge position. Cross-correlating that anchor with the other transmitted response polynomials estimates every coefficient of the signing secret directly; after Shuttle’s matching stretch and compression factors cancel, the signal is approximately $2s_i$.

We give a streaming chosen-message attack using only ordinary valid signatures. Rounding the empirical covariances recovers s_1 . The omitted error vector is then computed exactly from the public relation $b = a_{\text{gen}} + A_{\text{gen}}s_1 + s'_2 \pmod{q}$. Public coefficient bounds, the key-generation norm window, and the public key provide an exact validation test. The resulting equivalent secret key signs arbitrary messages.

End-to-end experiments used the unmodified submitted reference signer and verifier. Shuttle-128 was broken for four independent keys using 150,000–225,000 signatures and 311–467 single-threaded CPU-seconds. Shuttle-256 was broken with 225,000 signatures in 740 CPU-seconds. Shuttle-512 was broken with 325,000 signatures in 2,208.998 CPU-seconds. Every queried signature was verified, and each recovered key produced an accepted signature on a fresh message. Peak resident memory was below 2.5 MB. All runs used a 3.2 GHz Intel Core i5-6500 with 16 GiB of RAM, placing the complete attack well within commodity desktop resources.

1 Introduction

Fiat–Shamir lattice signatures add a secret-dependent shift to a wide random mask. Their rejection or transition rule has one central privacy obligation: the public response distribution must be independent of the secret. A sign error in that rule is especially dangerous because honest signatures then become repeated noisy measurements of the signing key.

Shuttle is an NGCC lattice-signature submission based on MLWE and MSIS [1]. It adapts iterative rejection sampling [3] into a two-output transition with no inner abort. A sparse challenge selects cyclic shifts of a stretched secret vector, and the transition chooses the sign of each shift. The submission claims that the accumulated response is close to a public discrete Gaussian, with the remaining approximation controlled by Rényi divergence.

The submitted transition does the opposite of its own probability definition. In the submission’s notation, $p_v(y)$ is the probability of returning $y - v$. Its interval test recognizes an event of probability $p_v(y)$, sets a flag to +1 on that event, and returns $y + \text{flag}v$. The portable C implementation makes the identical update. Thus it returns $y + v$ where the analysis requires $y - v$, and vice versa. The construction published separately by the Shuttle authors explicitly returns $y - v$ on the interval event [2]. This provides a direct cross-check of the branch convention used by the NGCC submission.

The resulting leak is visible in second moments. The correct transition must cancel the covariance introduced by adding a signed v . Reversing its output sign doubles that covariance instead. Shuttle makes this exploitable without learning the sampler’s internal signs: the full secret is $(1, s_1, s'_2)$, and the response block corresponding to the known component 1 is transmitted. It exposes a noisy copy of each internal sign at the corresponding challenge position. The other transmitted blocks then expose the aligned coefficients of s_1 .

Our contributions are:

- We identify the reversed branch in both the NGCC specification and the submitted portable implementation, and contrast it with the correctly oriented transition in the authors’ public ePrint.
- We derive the rank-one covariance term caused by the reversal and turn it into a coefficient-wise estimator over $\mathbb{Z}[X]/(X^n + 1)$.
- We give a streaming recovery algorithm that stores only $3n$ signed accumulators and uses no signing randomness, side channel, malformed input, or hash weakness.
- We complete the recovered s_1 to an equivalent signing key using the public key alone and validate it with all public key-generation bounds.
- We demonstrate accepted fresh-message forgeries against all three submitted parameter sets on an ordinary four-core 2015 desktop CPU, using far fewer than one million signatures.

The attack is on the version submitted to NGCC. It does not apply to an implementation of the correctly oriented transition described in [2].

2 Relevant parts of Shuttle

Let

$$\mathcal{R}_q = \mathbb{Z}_q[X]/(X^n + 1).$$

The public key contains a seed defining $a_{\text{gen}} \in \mathcal{R}_q^m$ and $A_{\text{gen}} \in \mathcal{R}_q^{m \times \ell}$, together with $b \in \mathcal{R}_q^m$. Key generation samples $s_1 \in \mathcal{R}^\ell$ and $s_2 \in \mathcal{R}^m$, rounds the initial public value, and absorbs the

rounding discrepancy into an adjusted error s'_2 . Consequently the encoded public key satisfies the exact relation

$$b = a_{\text{gen}} + A_{\text{gen}}s_1 + s'_2 \pmod{q}. \quad (1)$$

The full signing secret used by the iterative sampler is

$$s = (1, s_1, s'_2).$$

2.1 Stretching, compression, and the response

Shuttle uses block-dependent integer factors $\alpha_1, \alpha_s, \alpha_e$. Define

$$\begin{aligned} \text{StretchS}(x_0, x_s, x_e) &= (\alpha_1 x_0, \alpha_s x_s, \alpha_e x_e), \\ \text{CompressY}(y_0, y_s, y_e) &= (\lfloor y_0/\alpha_1 \rfloor, \lfloor y_s/\alpha_s \rfloor, \lfloor y_e/\alpha_e \rfloor). \end{aligned}$$

The shifted-floor convention gives the exact translation identity

$$\text{CompressY}(y + \text{StretchS}(x)) = \text{CompressY}(y) + x \quad (x \text{ integral}). \quad (2)$$

The signer samples a wide discrete-Gaussian vector y , computes a Fiat–Shamir challenge

$$c = \sum_{j \in J} X^j, \quad |J| = \tau,$$

and processes the indices $j \in J$ in increasing order. For each index it uses the shift

$$v_j = \text{StretchS}(s)X^j$$

and chooses a hidden $\epsilon_j \in \{-1, +1\}$. Before compression, the final state has the form

$$\tilde{z} = y + \sum_{j \in J} \epsilon_j v_j.$$

Equation (2) makes the public response exactly

$$z = \text{CompressY}(y) + s \sum_{j \in J} \epsilon_j X^j. \quad (3)$$

Only the first $1 + \ell$ polynomials, denoted z_1 , are serialized directly; the m error-block polynomials are represented through a hint. The attack needs only z_1 and the public challenge seed.

For every $j \in J$, Equation (3) gives

$$z_{1,0}[j] = \lfloor y_0[j]/\alpha_1 \rfloor + \epsilon_j, \quad (4)$$

$$z_{1,p}[j + d]_{\text{neg}} = \lfloor y_p[j + d]_{\text{neg}}/\alpha_s \rfloor + \epsilon_j s_{1,p}[d] + \text{other shifts}, \quad (5)$$

where $1 \leq p \leq \ell$, and the signed negacyclic access is

$$x[k]_{\text{neg}} = \begin{cases} x[k], & 0 \leq k < n, \\ -x[k - n], & n \leq k < 2n. \end{cases}$$

2.2 The intended transition

For a single shift v , Shuttle defines

$$S_v(y) = \sum_{i=0}^{\infty} (-1)^i \frac{\rho_r(y + iv)}{\rho_r(y)}$$

and

$$p_v(y) = \begin{cases} S_v(y), & \langle y, v \rangle \geq 0, \\ 1 - S_v(-y), & \langle y, v \rangle < 0. \end{cases} \quad (6)$$

Both the NGCC prose and the authors' ePrint define $p_v(y)$ as the probability of returning $y - v$; $y + v$ has probability $1 - p_v(y)$ [1, 2]. The finite alternating sum is implemented by drawing $U \in (0, 1]$ and testing whether $2r^2 \log U$ lies in a union of intervals whose total probability is $p_v(y)$.

After normalizing the sign of v so that $\langle y, v \rangle > 0$, let $B(y, v, U)$ be $+1$ when this interval test succeeds and -1 otherwise. The correct transition is

$$y - Bv. \quad (7)$$

Equivalently, the ePrint writes the event bit as $b \in \{0, 1\}$ and returns $y + (-1)^b v$, so membership ($b = 1$) returns $y - v$ [2].

2.3 The submitted sign reversal

Algorithm 22 of the NGCC specification initializes a flag to -1 , sets it to $+1$ when the same interval test succeeds, and returns

$$y + Bv. \quad (8)$$

The submitted C implementation also updates every coefficient by $z \leftarrow z + Bv$. Thus Equations (7) and (8) are exact opposites for the same random event.

This is not a harmless renaming of v . The probability in Equation (6) depends on the oriented inner product, and its purpose is to counteract the selected shift. The submitted update reinforces it.

3 Why the reversal creates covariance

The leak can be seen without evaluating the alternating series. Consider one transition with input Y of mean zero and covariance Σ . Let $E \in \{-1, +1\}$ include both the sign normalization of v and the outcome of the interval test. The intended output is

$$Z_- = Y - Ev,$$

whereas the submitted output for the same event is

$$Z_+ = Y + Ev.$$

Write $\mu = \mathbb{E}[EY]$. Since $E^2 = 1$,

$$\mathbb{E}[Z_- Z_-^T] = \Sigma - \mu v^T - v \mu^T + v v^T, \quad (9)$$

$$\mathbb{E}[Z_+ Z_+^T] = \Sigma + \mu v^T + v \mu^T + v v^T. \quad (10)$$

The intended transition is designed to preserve the Gaussian up to a tiny pointwise error. In the idealized exact case, $\mathbb{E}[Z_- Z_-^T] = \Sigma$. The resulting matrix identity $\mu v^T + v \mu^T = v v^T$ itself

forces $\mu = v/2$: multiplication by any vector orthogonal to v first shows that μ is parallel to v , and comparison along v fixes the scalar. Substitution into Equation (10) yields

$$\mathbb{E}[Z_+ Z_+^T] = \Sigma + 2vv^T. \quad (11)$$

For the approximate discrete transition, the same calculation has an error controlled by the deviation of the intended output from the target Gaussian. The submission claims that deviation is negligible at its parameters; it cannot cancel the leading $2vv^T$ term.

For $v_j = \text{StretchS}(s)X^j$, the first block of v_j has the single nonzero coefficient α_1 at position j . Its cross-covariance with coefficient $j + d$ of secret block p is therefore

$$2\alpha_1\alpha_s s_{1,p}[d].$$

The public compression divides the two blocks by the same factors. Thus the observable cross-covariance in Equations (4) and (5) is

$$\mathbb{E}[z_{1,0}[j] z_{1,p}[j + d]_{\text{neg}} \mid j \in J] \approx 2s_{1,p}[d]. \quad (12)$$

Equation (12) motivates the estimator used below. In the complete signing process, other challenge shifts, compression and rounding, later transitions, and the final public norm gate introduce additional terms. We expect cross-terms associated with unrelated random challenge locations to average toward zero; rather than assuming that this approximation is exact, Section 5 measures the resulting coefficient scale directly. Across all tested parameter sets the empirical multiplier remains close to the predicted value 2, while the corrected-sign control of Section 5.4 removes the signal.

4 Key recovery and forgery

The complete attack has four stages: collect signatures, estimate s_1 , derive s'_2 from the public key, and serialize an equivalent signing key.

4.1 Streaming covariance estimator

Initialize signed 64-bit accumulators $C_{p,d} = 0$ for $1 \leq p \leq \ell$ and $0 \leq d < n$, and an observation counter $M = 0$. For each ordinary signature:

1. Verify it with the submitted public verifier and discard it on any failure.
2. Decode the challenge seed and the transmitted response z_1 , then recompute the public binary challenge c .
3. For every j with $c[j] = 1$, update

$$C_{p,d} += z_{1,0}[j] z_{1,p}[j + d]_{\text{neg}} \quad \text{for all } p, d, \quad (13)$$

and increment M .

At a checkpoint, set

$$\hat{s}_{1,p}[d] = \left\lfloor \frac{C_{p,d}}{2M} \right\rfloor, \quad (14)$$

clamped to the public encoding interval for a secret coefficient. The scale 2 is fixed by Equation (11); it is not fitted from the unknown key. The estimator stores $O(\ell n)$ words. The straightforward update cost is $O(Q\tau\ell n)$ for Q signatures. FFT-based batched correlation could reduce this cost, but was unnecessary for the reported laptop runs.

4.2 Public completion and exact validation

Once $\widehat{s}_1$ is available, expand a_{gen} and A_{gen} from the public seed and compute

$$\widehat{s}'_2 = \text{center}_q(b - a_{\text{gen}} - A_{\text{gen}}\widehat{s}_1). \quad (15)$$

This is public computation. A candidate is accepted only if

- every coefficient of $\widehat{s}_1$ and $\widehat{s}'_2$ is inside the submitted secret-key encoding range;
- $\text{StretchS}(1, \widehat{s}_1, \widehat{s}'_2)$ lies inside the submitted two-sided key-generation norm window; and
- Equation (1) holds exactly.

These checks use no original secret information. In the reported experiments, public validation first succeeded at the same checkpoint at which all coefficients of s_1 matched the experimental ground truth. An accepted fresh-message signature produced by the completed key is the final end-to-end validation.

4.3 Equivalent-key forgery

The serialized Shuttle secret key contains the public seed and b , an auxiliary signing seed K , the public-key hash, and (s_1, s'_2) . After Equations (14) and (15), choose any new K , recompute the public-key hash, and encode these values in the documented secret-key format. This key need not equal the original byte string. It satisfies the same public relation and all signing bounds, so the public signing algorithm can sign an arbitrary message. Verification uses only the original public key.

The demonstrated attack operates in the EUF-CMA setting: it obtains valid signatures on distinct messages and subsequently produces an accepted signature on a fresh message that was not queried before.

5 Parameters and experiments

5.1 Submitted parameter sets

Table 1 lists the quantities relevant to the attack. All three sets use $\ell = 3$ and Gaussian width $r = 825$.

Table 1: Relevant parameters of the submitted Shuttle instances.

Instance	n	q	m	τ	α_1	α_s	B_s	Sig. bytes
Shuttle-128	256	15361	3	42	90	10	9	1183
Shuttle-256	512	61441	2	58	135	5	10	2417
Shuttle-512	1024	59393	2	115	144	3	10	5001

5.2 Methodology

We used the portable C reference implementation from the public NGCC archive, whose SHA-256 digest is

09253fd33c2215098177991db3375cb94e4e4a73a65b019783edabefa5bd8a62.

The scheme sources were not modified. They were compiled with GCC 13.3.0 at `-O3` and `-march=native`. The attack ran as one process and did not use worker threads. The machine had an Intel Core i5-6500 at 3.2 GHz, four physical cores, and 16 GiB of RAM; CPU times therefore describe one ordinary core and are more reproducible than wall times under concurrent system load.

For each query we signed a distinct 32-byte message using the implementation’s explicit-randomness API, verified the result, decoded it through the public signature decoder, and applied Equation (13). Checkpoints were taken every 25,000 signatures. Recovery used the fixed divisor $2M$. A least-squares scale computed using the experimental ground truth was printed only as a diagnostic after the estimate; it was never an attack input.

Success required all public completion checks and an accepted signature on the fresh message

fresh message signed with the statistically recovered key.

Every honest query and every reported forgery was checked by the unmodified submitted verifier.

5.3 Results

Table 2 gives the first checkpoint at which the direct rounded estimate passed public validation. A diagnostic regression slope between average covariance and the known experimental secret stayed between 1.95 and 2.03 during the runs. Its proximity to 2 supports the scale predicted by Equation (12); the actual attack always used the fixed value 2 and never the diagnostic fit.

Table 2: End-to-end equivalent-key recovery and fresh-message forgery.

Instance	Key tag	Signatures	Observations	CPU s	Forgery
Shuttle-128	42	200,000	8,400,000	413.175	accept
Shuttle-128	43	150,000	6,300,000	310.879	accept
Shuttle-128	44	225,000	9,450,000	467.112	accept
Shuttle-128	45	175,000	7,350,000	362.885	accept
Shuttle-256	42	225,000	13,050,000	739.804	accept
Shuttle-512	42	325,000	37,375,000	2208.998	accept

The four Shuttle-128 trials used independent deterministic key seeds. Direct rounding recovered all $3n = 768$ coefficients and public completion recovered all 768 or 512 error coefficients, as appropriate. The Shuttle-256 and Shuttle-512 trials recovered all 1536 and 3072 coefficients of s_1 and all 1024 and 2048 coefficients of s'_2 , respectively. The generated forgery lengths were exactly 1183, 2417, and 5001 bytes.

Convergence was smooth rather than a rare event. For the Shuttle-512 trial, the exact secret-coefficient counts at 25,000-signature intervals were 2189, 2652, 2856, 2965, 3016, 3045, 3061, 3066, 3068, 3069, 3070, 3070, and 3072 out of 3072. Incomplete checkpoints failed the public validation test. The fitted scale stayed within a few percent of 2 throughout.

The attack state itself consists principally of $3n$ 64-bit accumulators: 6 KiB, 12 KiB, and 24 KiB for the three levels. The measured peak resident set, including the submitted signer, verifier, decoding buffers, and process runtime, stayed below 2.5 MB. Transcript storage is unnecessary.

5.4 Corrected-sign control

To isolate the cause, we made one experimental change to a separate copy of the Shuttle-128 reference implementation: the final transition update was changed from $z \leftarrow z + Bv$ to $z \leftarrow z - Bv$. No attack or verifier code was changed. All 25,000 generated signatures verified,

but the ground-truth diagnostic slope fell from approximately 2 to 0.03799 and Equation (14) recovered only 348 of 768 coefficients. The candidate failed public validation. This control is consistent with the corrected transition suppressing the rank-one covariance term and directly links the observed key recovery to the sign reversal.

6 Scope, cause, and repair

The reversed branch direction is present in both the NGCC specification pseudocode and the submitted reference implementation. The attack is not caused by the placeholder SM3-based XOF, floating-point rounding, compiler behavior, timing leakage, or malformed decoding. Replacing the XOF with a random oracle leaves the covariance mechanism intact.

The minimal conceptual repair is to apply the interval event to the opposite shift: after the sign normalization used by the submitted algorithm, update the state by $y - Bv$ instead of $y + Bv$. This matches the definition of $p_v(y)$ and the transition in [2]. Changing that sign changes known-answer vectors and requires a fresh correctness, distribution, and constant-time review; implementers should not treat it as an isolated source patch without regenerating and independently validating the submission.

The attack does not claim a break of the correctly oriented construction in the separately published ePrint paper. That construction has a different public parameterization and states the branch direction consistently. Our result is specific to the public NGCC specification and the code submitted with it.

7 Disclosure and prior-art check

Before this manuscript was prepared, we searched the public NGCC report index, the pqc-forum web archive through general web search, the IACR ePrint index, and a contemporary roundup of NGCC findings [4, 5]. On 25 September 2026 the NGCC index listed Shuttle (sign-23) as “No report,” and the roundup did not mention Shuttle. We found the authors’ ePrint [2], which is directly relevant because it specifies the correct branch direction, but it does not report the reversed NGCC implementation or this key-recovery attack. This is a time-bounded search, not proof of novelty; human review remains pending.

8 Conclusion

In the NGCC submission, the interval event associated in the specification with the $y - v$ branch is applied by Algorithm 22 and the reference implementation to the $y + v$ update. In the idealized one-transition analysis, this sign reversal replaces covariance cancellation with an additional $2vv^T$ secret-dependent term. The known constant component of Shuttle’s secret makes the term directly observable in ordinary signatures. A simple streaming correlation recovers the complete signing secret, the public key determines the omitted error, and the resulting equivalent key forges fresh messages for every submitted parameter set. The largest demonstrated run uses well below one million signatures, one CPU core, and negligible attack memory.

References

- [1] L. Chen, Y. Ming, J. Zhang, C. Zhang, Z. Liu, G. Tang, S. Gao, P. Chen, Y. Sun, T. Wang, M. Mei, and J. Hou, *Shuttle: A Lattice Based Digital Signature Scheme—Algorithm Specification and Supporting Documentation*, NGCC submission, 30 June 2026.

- [2] Y. Ming, J. Zhang, Z. Liu, G. Tang, P. Chen, Y. Sun, S. Gao, C. Zhang, and L. Chen, *Towards Practical Iterative Rejection Sampling: A Compact and Efficient Signature over Module Lattices*, IACR Cryptology ePrint Archive, Report 2026/1991, 2026. <https://eprint.iacr.org/2026/1991>
- [3] J. G rtner, *Compact Lattice Signatures via Iterative Rejection Sampling*, Advances in Cryptology–CRYPTO 2025, LNCS 16000, pp. 514–547, 2025.
- [4] M.-J. O. Saarinen, *NGCC Reports*, public report index, accessed 25 September 2026. <https://ngcc.dev/reports/index.html>
- [5] M. Ivezic, *China’s Next-Generation Crypto Candidates Drew 104 Public Findings in Three Days*, PostQuantum.com, 23 September 2026. <https://postquantum.com/security-pqc/china-ngcc-candidate-flaws/>